

Software Requirements Specification

EN2H Booking Platform API

Version 1.0 | Prepared for the EN2H Software Engineer Intern (NestJS) Technical Assignment

Author: Shreyanshu Maske | Date: July 2026

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Intended Audience
- 1.4 Definitions and Acronyms
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 Assumptions and Dependencies

3. Data Model

4. Functional Requirements

- 4.1 Authentication
- 4.2 Service Management
- 4.3 Booking Management
- 4.4 Business Rules

5. External Interface Requirements (REST API)

6. Non-Functional Requirements

7. Appendix: Error Response Format

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for the EN2H Booking Platform API — a backend REST service that allows a business to manage a catalog of services and accept and manage customer bookings against those services. It was produced as part of the EN2H Software Engineer Intern (NestJS) technical assignment and documents the system as actually implemented.

1.2 Scope

The system provides:

- User registration and JWT-based authentication (access + refresh tokens).
- CRUD management of bookable services, restricted to authenticated staff users.
- Creation and lifecycle management of customer bookings against those services, with booking creation open to unauthenticated customers.
- Enforcement of booking business rules (service existence/active status, no past-dated bookings, valid status transitions, no duplicate slots).

Out of scope: a frontend/UI, payment processing, notification/email delivery, and multi-tenant organization support.

1.3 Intended Audience

Technical reviewers evaluating the assignment; any engineer who needs to operate, extend, or integrate with the API.

1.4 Definitions and Acronyms

Term	Definition
JWT	JSON Web Token — signed token used to authenticate API requests.
Access Token	Short-lived JWT (default 15 minutes) sent as a Bearer token to authorize requests.
Refresh Token	Longer-lived JWT (default 7 days), exchanged for a new access/refresh pair; stored server-side as a hash and rotated on use.
DTO	Data Transfer Object — a validated shape for request/response payloads.
ORM	Object-Relational Mapper (TypeORM in this system).
Slot	The combination of a service, a booking date, and a booking time.

1.5 References

- NestJS documentation — <https://docs.nestjs.com>
- TypeORM documentation — <https://typeorm.io>
- OpenAPI/Swagger specification, as exposed by this API at `/api/docs-json`

2. Overall Description

2.1 Product Perspective

The system is a standalone, self-contained REST API built with NestJS (TypeScript) and PostgreSQL (via TypeORM). It has no dependency on external third-party services beyond its own database; JWT signing/verification and password hashing are performed in-process. It is designed to be consumed by any HTTP client — a web frontend, mobile app, or another backend service.

2.2 Product Functions

- Register and authenticate users; issue and rotate JWT access/refresh tokens; revoke sessions on logout.
- Create, update, delete, list, and retrieve services.
- Create bookings (public), list/retrieve/update-status/cancel bookings (authenticated).
- Validate all input and return structured error responses.
- Expose interactive API documentation via Swagger.

2.3 User Classes and Characteristics

User Class	Description	Access Level
Staff / Authenticated User	Registers an account, manages the service catalog, and manages the lifecycle of bookings.	Full access to Service and Booking management endpoints via a valid JWT access token.
Customer	An end customer who books a service. Does not need an account.	Can only call <code>POST /bookings</code> ; cannot list, view, or modify existing bookings.

2.4 Operating Environment

- Runtime: Node.js 20+, NestJS 11.
- Database: PostgreSQL 16 (also verified with PostgreSQL 18 on the deployed environment).
- Deployable via Docker (multi-stage build) or directly with Node; verified running both locally via Docker Compose and on Render.com.

2.5 Design and Implementation Constraints

- Must be implemented in NestJS + TypeScript, per the assignment brief.
- Schema changes are managed exclusively through TypeORM migrations — `synchronize` is disabled, so the database schema does not implicitly drift from entity definitions.
- All secrets (DB credentials, JWT signing secrets) are supplied via environment variables, never hard-coded or committed to source control.

2.6 Assumptions and Dependencies

- Listing, viewing, updating the status of, and cancelling bookings are treated as staff-facing operations requiring authentication, since booking records contain customer PII (email, phone). The assignment brief only explicitly requires booking *creation* to be unauthenticated.
- A booking cannot be created against a service whose `isActive` flag is `false`.
- Any authenticated user may manage any service — there is no per-user service ownership model in this version.
- Duplicate-booking prevention is scoped to the tuple `(serviceId, bookingDate, bookingTime)` and ignores bookings that are already `CANCELLED` (a cancelled slot can be re-booked).

3. Data Model

User

Field	Type	Constraints
id	UUID	Primary key, server-generated
email	string	Unique, not null
passwordHash	string	bcrypt hash, not null
name	string	Not null
createdAt / updatedAt	timestamp	Auto-managed

RefreshToken

Field	Type	Constraints
id	UUID	Primary key
userId	UUID	Foreign key → User, cascade delete
tokenHash	string	SHA-256 hash of the issued refresh token (see §6, Security)
expiresAt	timestampz	Not null
revoked	boolean	Default false

Service

Field	Type	Constraints
id	UUID	Primary key
title	string	Not null
description	text	Not null
duration	integer	Minutes, not null, ≥ 1
price	decimal(10,2)	Not null, ≥ 0
isActive	boolean	Default true
createdAt / updatedAt	timestamp	Auto-managed

Booking

Field	Type	Constraints
id	UUID	Primary key
customerName	string	Not null
customerEmail	string	Not null, valid email format
customerPhone	string	Not null, validated phone pattern
serviceId	UUID	Foreign key → Service, ON DELETE RESTRICT
bookingDate	date	Not null, cannot be in the past
bookingTime	time	Not null, format HH:mm
status	enum	PENDING CONFIRMED CANCELLED COMPLETED — default PENDING
notes	text	Nullable
createdAt / updatedAt	timestamp	Auto-managed

A unique index on `(serviceId, bookingDate, bookingTime)` enforces the no-double-booking rule at the database level, in addition to an application-level check that returns a clean `409 Conflict`.

4. Functional Requirements

4.1 Authentication

ID	Requirement
FR-1.1	The system shall allow a new user to register with email, password, and name. Duplicate emails shall be rejected with <code>409 Conflict</code> .
FR-1.2	The system shall allow a registered user to log in with email and password, returning an access token and a refresh token on success, and <code>401 Unauthorized</code> on failure.
FR-1.3	The system shall allow a valid, non-revoked refresh token to be exchanged for a new access/refresh token pair (rotation), invalidating the token that was exchanged.
FR-1.4	The system shall allow an authenticated user to log out, revoking all of their active refresh tokens.

4.2 Service Management

ID	Requirement
FR-2.1	An authenticated user shall be able to create a service (title, description, duration, price, isActive).
FR-2.2	An authenticated user shall be able to update any field of an existing service.
FR-2.3	An authenticated user shall be able to delete a service. Deletion is blocked (foreign-key restrict) if bookings reference it.
FR-2.4	An authenticated user shall be able to list all services, with pagination and an optional <code>isActive</code> filter.
FR-2.5	An authenticated user shall be able to retrieve a single service by ID, returning <code>404</code> if not found.

4.3 Booking Management

ID	Requirement
FR-3.1	Any client, authenticated or not, shall be able to create a booking by supplying customer details, a service ID, a date, and a time.
FR-3.2	An authenticated user shall be able to list all bookings, with pagination, an optional status filter, and a search over customer name/email.
FR-3.3	An authenticated user shall be able to retrieve a single booking by ID.

FR-3.4	An authenticated user shall be able to change a booking's status, subject to the transition rules in §4.4.
FR-3.5	An authenticated user shall be able to cancel a booking, subject to the same transition rules.

4.4 Business Rules

ID	Rule	Enforced As
BR-1	A booking must reference an existing service.	<code>404 Not Found</code> if the service does not exist.
BR-2	A booking cannot be created against an inactive service.	<code>400 Bad Request</code> .
BR-3	Booking dates cannot be in the past.	<code>400 Bad Request</code> , enforced by a custom <code>class-validator</code> decorator and re-checked at the service layer.
BR-4	A cancelled booking can never be marked completed.	Fixed status transition map: <code>PENDING</code> → <code>{CONFIRMED, CANCELLED}</code> ; <code>CONFIRMED</code> → <code>{COMPLETED, CANCELLED}</code> ; <code>CANCELLED</code> and <code>COMPLETED</code> are terminal. Invalid transitions return <code>400 Bad Request</code> .
BR-5	Only authenticated users may manage services.	JWT guard on all <code>/services</code> routes.
BR-6	Customers may create bookings without authentication.	<code>POST /bookings</code> carries no guard.
BR-7 (bonus)	No two bookings may occupy the same service/date/time slot (unless the earlier one was cancelled).	Service-layer check (<code>409 Conflict</code>) plus a database unique index as a safety net.

5. External Interface Requirements (REST API)

All routes are served under the `/api/v1` prefix. Full interactive documentation, including request/response schemas, is published via Swagger at `/api/docs` (JSON at `/api/docs-json`).

Method	Path	Auth	Description
POST	/auth/register	—	Register a new user
POST	/auth/login	—	Log in, receive token pair
POST	/auth/refresh	Refresh token	Rotate token pair
POST	/auth/logout	Bearer	Revoke active refresh tokens
POST	/services	Bearer	Create a service
GET	/services	Bearer	List services (paginated, filter by isActive)
GET	/services/:id	Bearer	Get a service
PATCH	/services/:id	Bearer	Update a service
DELETE	/services/:id	Bearer	Delete a service
POST	/bookings	—	Create a booking
GET	/bookings	Bearer	List bookings (paginated, filter by status, search)
GET	/bookings/:id	Bearer	Get a booking
PATCH	/bookings/:id/status	Bearer	Update booking status
PATCH	/bookings/:id/cancel	Bearer	Cancel a booking

6. Non-Functional Requirements

Security

- Passwords are hashed with bcrypt (never stored or logged in plaintext).
- Refresh tokens are never stored in plaintext server-side; they are hashed with SHA-256 before persistence. (bcrypt was deliberately not used here — bcrypt truncates input at 72 bytes, and JWTs routinely exceed that, which was found during testing to let any refresh token issued to a user match any other due to a shared byte prefix. SHA-256 is the correct, standard approach for hashing high-entropy opaque tokens.)
- Refresh tokens rotate on every use and are individually revocable; a rotated token cannot be replayed.
- All input is validated and unknown properties are rejected (`whitelist` + `forbidNonWhitelisted`), reducing mass-assignment risk.
- Secrets are supplied exclusively via environment variables; `.env` is git-ignored and no credentials are committed to source control.

Reliability & Data Integrity

- Schema is managed via versioned migrations, not runtime auto-sync, so production schema changes are explicit and reviewable.
- Foreign key from Booking → Service uses `ON DELETE RESTRICT` , preventing orphaned bookings.
- The duplicate-booking rule is enforced at both the application layer (clean error message) and the database layer (unique index), so it holds even under concurrent requests.

Maintainability

- Code is organized into one NestJS module per bounded context (auth, users, services, bookings) plus a shared common/ module for cross-cutting concerns (filters, decorators, validators, pagination DTO).
- 22 automated unit tests cover the business-rule-critical paths in `BookingsService` and `AuthService` .

Usability / Developer Experience

- Full interactive Swagger documentation with bearer-auth support.
- Consistent, structured JSON error responses across the entire API.

Portability / Deployability

- Multi-stage Dockerfile and Docker Compose file for local development.
- A Render Blueprint (`render.yaml`) provisions the web service and a managed PostgreSQL database together, with secrets auto-generated rather than committed.

7. Appendix: Error Response Format

Every error response, regardless of source, is normalized by a global exception filter to the following shape:

Field	Type	Description
statusCode	number	HTTP status code
message	string string[]	Human-readable error message(s); validation errors return an array
error	string	HTTP status text, e.g. "Bad Request"
timestamp	string (ISO 8601)	When the error occurred
path	string	The request path that produced the error